Luckily, you can easily switch back to the Newtonsoft.Json library instead. Install the Microsoft.AspNetCore.Mvc.NewtonsoftJson package into your project and update the `AddControllers()` method in Program.cs to the following:

```
builder.Services.AddControllers()
    .AddNewtonsoftJson();
```

This will switch ASP.NET Core's formatters to use Newtonsoft.Json behind the scenes, instead of System.Text.Json. For more details on the differences between the libraries, see Microsoft's article "Compare Newtonsoft.Json to System.Text.Json, and migrate to System.Text.Json": http://mng.bz/0mRJ. For more advice on when to switch to the Newtonsoft.Json formatter, see the section "Add Newtonsoft.Json-based JSON format support" in Microsoft's "Format response data in ASP.NET Core Web API" documentation: http://mng.bz/zx11.

You can add XML output to your application by adding an output formatter. You configure your application's formatters in Program.cs by customizing the `IMvcBuilder` object returned from `AddControllers()`. To add the XML output formatter, use the following:

```
services.AddControllers()
    .AddXmlSerializerFormatters();
```

NOTE Technically, this also adds an XML input formatter, which means your application can now receive XML in requests too. Previously, sending a request with XML in the body would respond with a `415 Unsupported Media Type` response. For a detailed look at formatters, including creating a custom formatter, see the documentation at http://mng.bz/e5gG.

With this simple change, your API controllers can now format responses as XML as well as JSON. Running the same request as shown in figure 20.7 with XML support enabled means the app will respect the `text/xml accept` header. The formatter serializes the `string` array to XML as requested instead of defaulting to JSON, as shown in figure 20.8.